

Prometheus & Grafana
Handbook
Volume 3: Advanced Prometheus

≡ VERIQTA ≡

SENIOR ENGINEER
EDITION



PROMETHEUS & GRAFANA ≡ HANDBOOK ≡

FROM BASICS TO PRODUCTION EXCELLENCE



Build. Monitor. Alert. Analyze.
Keep Your Systems
Healthy & Reliable.



PERFORMANCE
OPTIMIZED



RELIABLE
& SCALABLE



SECURE
BY DESIGN



DISASTER
RESILIENT



ENGINEERED
FOR OPERATIONS



THE COMPLETE GUIDE TO MODERN OBSERVABILITY WITH PROMETHEUS & GRAFANA

1. ENTERPRISE MONITORING ARCHITECTURE

KEY IDEA

Great monitoring isn't just about collecting metrics. It's about turning data into actionable insight at scale.



WHAT IS ENTERPRISE MONITORING?

Enterprise monitoring is the practice of collecting, processing, storing, and visualizing telemetry data across all systems, environments, and business services to ensure reliability, performance, and availability at scale.

It powers faster incident response, better decisions, and happier users.

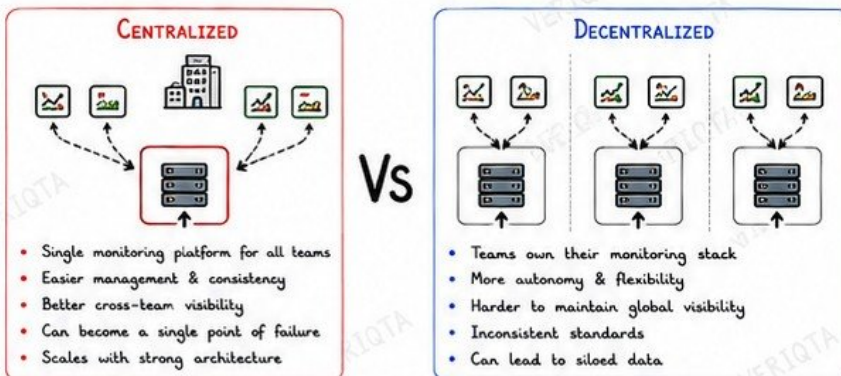
MONITORING AT SCALE

- ✓ Thousands of servers & containers
- ✓ Millions of metrics per second
- ✓ Multi-team, multi-tenant environments
- ✓ High availability & fault tolerance
- ✓ Global (multi-region) visibility
- ✓ Long-term retention & cost control

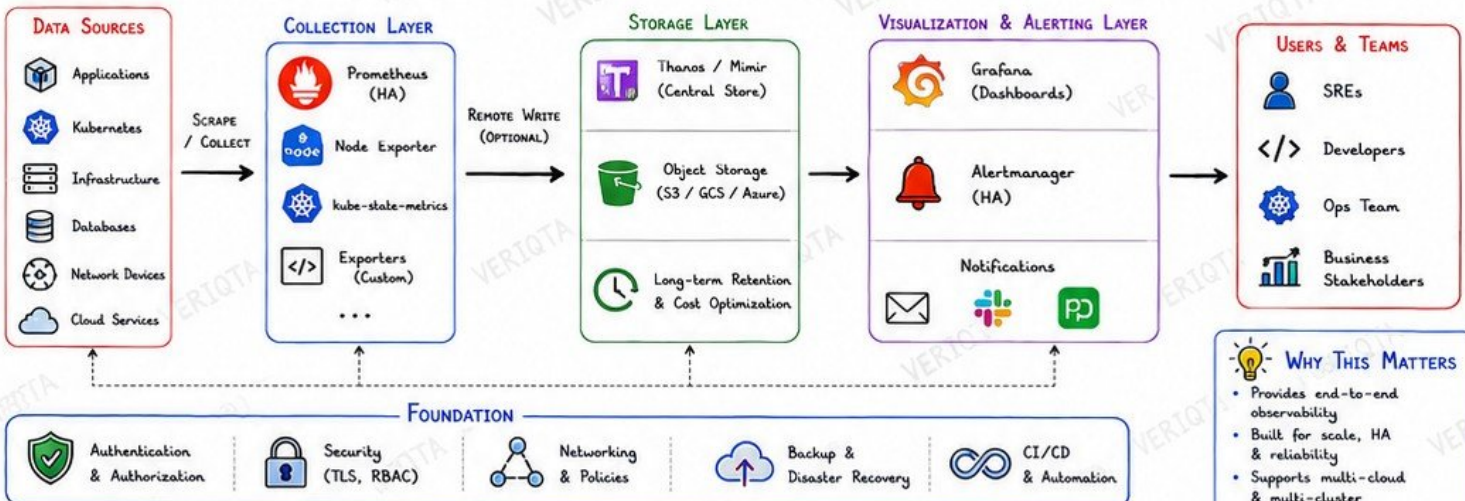
LAYERS OF OBSERVABILITY

USER EXPERIENCE	Real user monitoring (Synthetic + RUM)
APPLICATION	Logs, Traces, Metrics (App performance)
INFRASTRUCTURE	Servers, Containers, Networking, DBs
PLATFORM	Kubernetes, Cloud, Middleware, Services
DATA SOURCES	Metrics, Logs, Traces, Events, Alerts

CENTRALIZED VS DECENTRALIZED MONITORING



ENTERPRISE REFERENCE ARCHITECTURE



WHY THIS MATTERS

- Provides end-to-end observability
- Built for scale, HA & reliability
- Supports multi-cloud & multi-cluster
- Enables better incident response
- Drives business continuity



REMEMBER: You can't fix what you can't see.
Design your monitoring like you design your systems – for scale, failure, and growth.



GOOD MONITORING = BUSINESS UPTIME, HAPPY USERS, AND PEACE OF MIND.

2. LARGE-SCALE PROMETHEUS

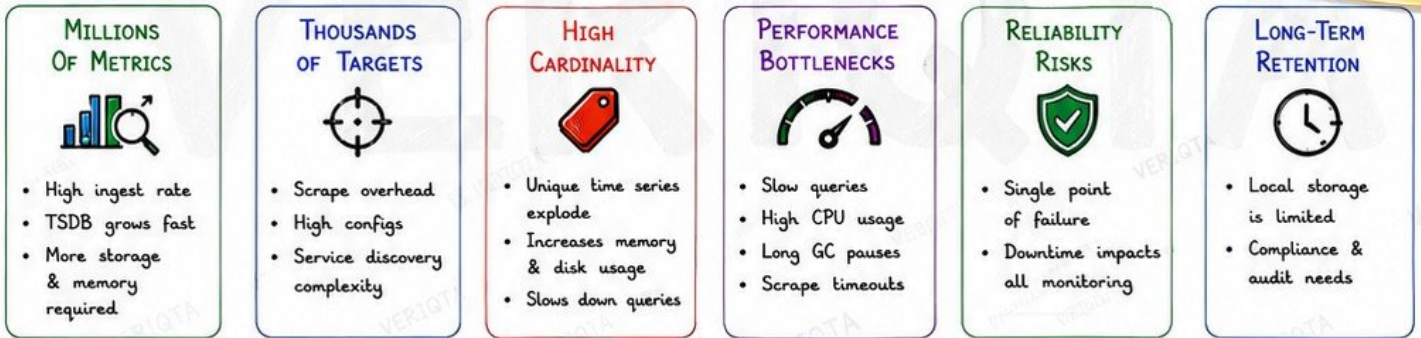
CHALLENGES BEYOND ONE PROMETHEUS SERVER

A single Prometheus server works great for small environments. At enterprise scale, it becomes a **bottleneck**.

KEY TAKEAWAY

Scale Prometheus horizontally, design for cardinality, optimize performance, and use the right architecture.

MAJOR CHALLENGES AT SCALE



WHAT SCALE LOOKS LIKE



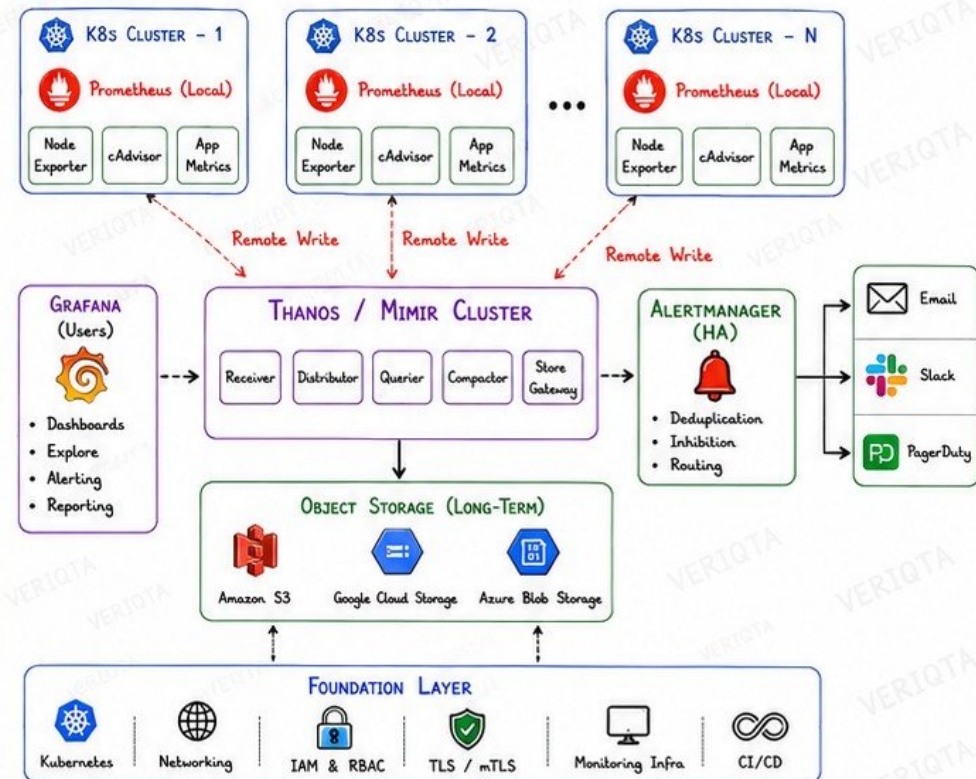
ENTERPRISE DEPLOYMENT EXAMPLE

- Multiple Kubernetes Clusters
Each cluster runs **local Prometheus** for metrics collection.
- Remote Write to Central Backend
Metrics are shipped to a scalable backend (Thanos / Mimir).
- Centralized Long-Term Storage
Object storage (S3 / GCS / Azure Blob) for durability.
- Global Query & Visualization
Grafana connects to centralized query layer.
- Alerting & Notifications
Alertmanager (HA) routes alerts to Email, Slack, PagerDuty.
- High Availability & Redundancy
Multiple replicas, zones, and regions for **resilience**.

RULE OF THUMB

- ✓ Keep cardinality under control.
- ✓ Use federation / remote write early.
- ✓ Store long-term data outside Prometheus.
- ✓ Design for HA from day one.
- ✓ Test at production scale before it hurts!

ENTERPRISE ARCHITECTURE EXAMPLE



BENEFITS OF SCALING RIGHT

- | | | | | | |
|-------------------------------------|-----------------------------|----------------------------|------------------------------|-----------------------|----------------------------|
| High availability & fault tolerance | Handles millions of metrics | Faster queries & better UX | Cost effective & predictable | Supports global teams | Built for long-term growth |
|-------------------------------------|-----------------------------|----------------------------|------------------------------|-----------------------|----------------------------|

3. SCALING STRATEGIES

MULTIPLE WAYS TO SCALE. CHOOSE WHAT FITS YOUR NEEDS.

KEY TAKEAWAY

There is no one-size-fit-all. Understand your scale, budget, team, and long-term goals before choosing a strategy.

1 HORIZONTAL SCALING

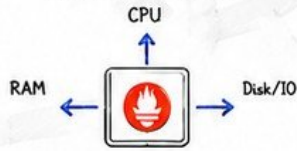
Add more Prometheus servers to handle more load.



- Distribute targets across servers
- Increases ingestion capacity
- Requires service discovery
- Good for high scrape load

2 VERTICAL SCALING

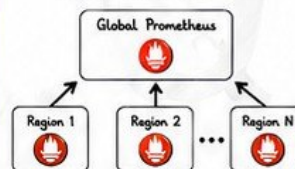
Increase resources (CPU/RAM/IO) of the Prometheus server.



- Simple to implement
- Limited by hardware limits
- Good for moderate scale
- Cost increases with limits

3 FEDERATION

Hierarchical scraping between Prometheus servers.



- Hierarchical aggregation
- Global view across regions
- Reduces load on global server
- Good for multi-region setups

4 SHARDING

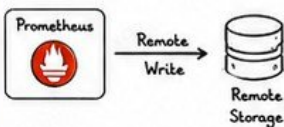
Split targets by labels or groups and scrape separately.



- Partition targets by label/tenant
- Parallel scraping & storage
- Improves performance
- Requires careful planning

5 REMOTE WRITE

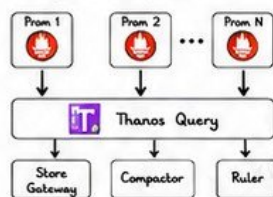
Send metrics to a remote backend for storage.



- Offload long-term storage
- Keep local for short retention
- Works with many backends
- Great for cost optimization

6 THANOS

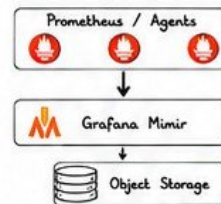
CNCF project for global view, long-term storage & HA.



- Global query across clusters
- Built-in long-term storage
- Deduplication & HA
- Battle-tested at scale

7 MIMIR

CNCF project for metrics at massive scale.



- Horizontally scalable backend
- Multi-tenant, auth & ACLs
- Cost-effective at high scale
- Designed for large orgs

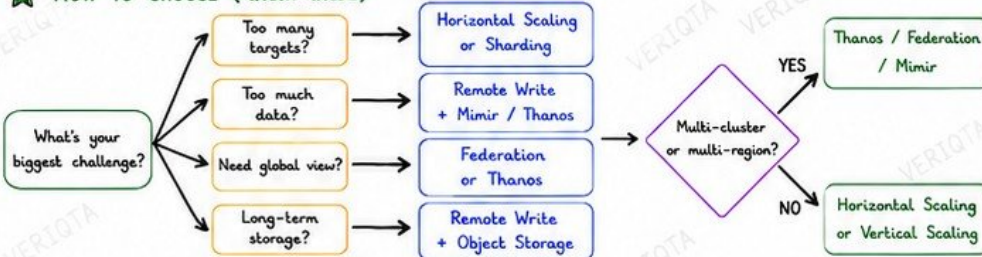
8 CHOOSE THE RIGHT STRATEGY

- ☒ How many targets do you have?
- ☒ How much data per day?
- ☒ How long do you need to retain?
- ☒ Do you operate multi-cluster/multi-region?
- ☒ What is your budget?
- ☒ What is your team's expertise?
- ☒ What are your performance requirements?



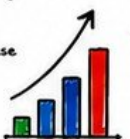
STRATEGY	SCALE	COMPLEXITY	COST	USE CASE	PROS	CONS
Horizontal Scaling	Medium - High	☆☆☆☆☆	\$\$	High scrape load	High throughput	More servers to manage
Vertical Scaling	Low - Medium	☆☆☆☆☆	\$\$\$	Moderate scale	Simple	Hardware limits
Federation	High	☆☆☆☆☆	\$\$\$	Multi-region / teams	Global view	Complex queries
Sharding	High	☆☆☆☆☆	\$\$\$	Large tenants / labels	Parallelism	Hard to re-shard
Remote Write	High	☆☆☆☆☆	\$ - \$\$\$	Long-term storage	Flexible, cost-saving	Dependent on backend
Thanos	High	☆☆☆☆☆	\$\$\$	Global HA & long-term	Feature rich	Operational overhead
Mimir	Very High	☆☆☆☆☆	\$ - \$\$	Massive scale, multi-tenant	Scalable, cost-effective	Newer ecosystem

★ HOW TO CHOOSE (QUICK GUIDE)



★ BEST PRACTICES AT SCALE

- ☒ Control cardinality early
- ☒ Use recording rules
- ☒ Set retention & limits wisely
- ☒ Monitor Prometheus itself
- ☒ Alert on saturation, not noise
- ☒ Test failover & recovery
- ☒ Automate everything



GOOD ARCHITECTURE TODAY = RELIABILITY TOMORROW.
SCALE **SMART**. SCALE **SUSTAINABLY**.

REMEMBER

Start simple, measure, then scale.
Monitoring is a product - iterate it!



4. STORAGE OPTIMIZATION

EFFICIENT STORAGE = LOWER COST, BETTER PERFORMANCE, LONGER RETENTION

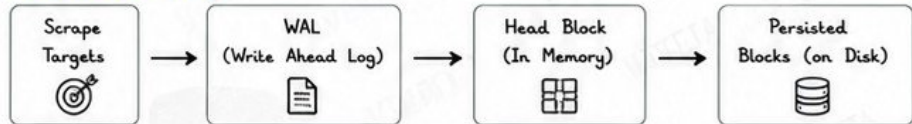
KEY IDEA

Store only what adds value, for as long as it adds value, and do it in the most efficient way.

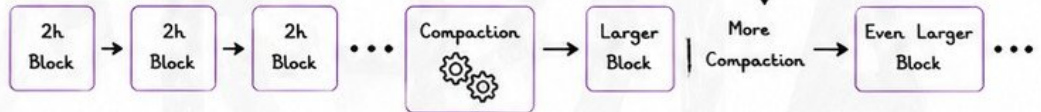
1 TSDB OPTIMIZATION

- Prometheus uses a **custom Time Series Database (TSDB)**
- Data is written to WAL, then persisted in blocks
- Optimize for write performance, query speed, and storage efficiency
- Key metrics: churn rate, head block size, compaction frequency
- Keep cardinality under control!

TSDB WRITE PATH



BLOCK LIFECYCLE



2 RETENTION POLICIES

- Control how long metrics are kept
- Set via `--storage.tsdb.retention.time` or `--storage.tsdb.retention.size`
- Time-based or size-based retention
- Use different retention for different environments

Examples

`--storage.tsdb.retention.time=30d`
`--storage.tsdb.retention.size=100GB`

3 BLOCK COMPACTION

- Small blocks are compacted into bigger blocks
- Reduces number of files
- Improves query performance
- Less overhead for object storage
- Controlled by Prometheus automatically

BEST PRACTICE

Don't disable compaction!
It keeps your storage healthy

4 COMPRESSION

- Prometheus uses XOR + Snappy compression
- Typical compression ratio: **1:5 to 1:10**
- High cardinality reduces compression
- Keep labels clean & consistent
- Avoid random IDs in labels

TIP

Lower cardinality =
Better compression = Lower cost

5 OBJECT STORAGE

- Store blocks in S3, GCS, Azure Blob for long-term retention
- Enables cheap, durable, scalable storage
- Required for Thanos / Mimir
- Supports global & multi-region setups

COMMON OPTIONS



AWS S3



Google Cloud Storage



Azure Blob Storage

6 COST OPTIMIZATION

- Right-size retention per environment
- Use longer retention in object storage
- Downsample old data (Thanos/Mimir)
- Drop unnecessary metrics/labels
- Monitor storage growth rate
- Set alerts for cost anomalies

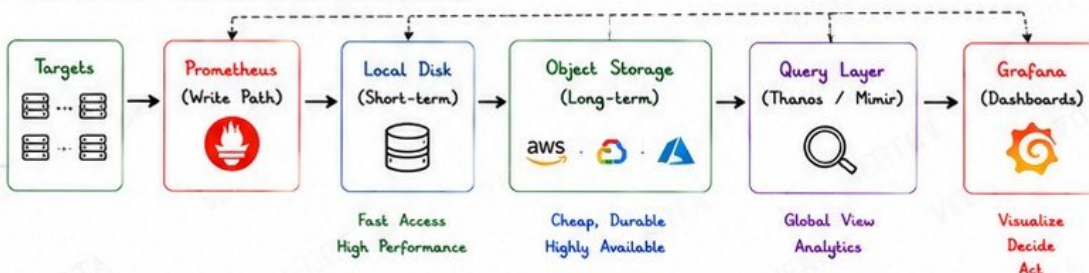
COST SAVING FORMULA

Lower Cardinality + Right Retention
+ Compression + Tiered Storage
= **Lower Cost**

7 BEST PRACTICES

- ✓ Limit cardinality (avoid labels like session_id, uuid)
- ✓ Use recording rules to reduce query load
- ✓ Set appropriate retention & review regularly
- ✓ Keep Prometheus version up to date
- ✓ Monitor TSDB metrics (disk, compactions, churn)
- ✓ Use object storage for long-term data
- ✓ Test restore & backup procedures
- ✓ Review & optimize storage periodically

END-TO-END STORAGE FLOW (ENTERPRISE)



GOLDEN RULE

You can't reduce what you don't understand.
Monitor your storage, trend it, and optimize continuously.

5. SECURITY BEST PRACTICES

SECURE MONITORING = PROTECT YOUR DATA, SYSTEMS & USERS

KEY IDEA

Monitoring systems hold critical data. Secure every layer – from network to storage.

1 AUTHENTICATION

- ✓ Always enable authentication
- ✓ Use strong, unique credentials
- ✓ Integrate with enterprise IdPs (OAuth2 / OIDC / LDAP / SAML)
- ✓ Disable anonymous access
- ✓ Rotate credentials regularly

Example

--web.enable-lifecycle=false

2 AUTHORIZATION

- ✓ Restrict access to endpoints
- ✓ Use fine-grained access control
- ✓ Limit who can query, manage & configure
- ✓ Audit access to sensitive data
- ✓ Principle of least privilege

Example

--web.external-url=https://

3 TLS ENCRYPTION

- ✓ Encrypt in transit (TLS)
- ✓ Use valid certificates (ACME / Let's Encrypt)
- ✓ Enable mTLS for service-to-service communication
- ✓ Redirect HTTP to HTTPS
- ✓ Keep TLS versions up to date

Example

tls_config:
insecure_skip_verify: false

4 NETWORK SECURITY

- ✓ Restrict access using firewalls
- ✓ Allow only required ports
- ✓ Use Private Networks / VPCs
- ✓ Block public exposure
- ✓ Use Network Policies (K8s)
- ✓ Enable IP allowlisting

COMMON PORTS

Prometheus: 9090
Alertmanager: 9093
Grafana: 3000

5 RBAC (ROLE BASED ACCESS)

- ✓ Use RBAC in Kubernetes
- ✓ Restrict Prometheus Operator permissions
- ✓ Limit access to namespaces
- ✓ Separate roles for dev, ops, viewers
- ✓ Regularly review & audit roles

Example

Role: prom-viewer
Can only view metrics, not modify.

6 SECRETS MANAGEMENT

- ✓ Store secrets in secure systems
- ✓ Use Kubernetes Secrets / SealedSecrets
- ✓ Use HashiCorp Vault / AWS Secrets Manager
- ✓ Never store secrets in config files
- ✓ Rotate secrets automatically

SECRETS TO PROTECT

- Basic Auth Credentials
- TLS Certificates & Keys
- Alertmanager Webhook Keys
- Remote Write Tokens

7 SECURE MONITORING ARCHITECTURE (PRINCIPLES)

- ★ Defense in depth
- ★ Least privilege access
- ★ Encrypt everything in transit
- ★ Separate monitoring network
- ★ Isolate storage
- ★ Audit & alert on access
- ★ Regular patching & updates

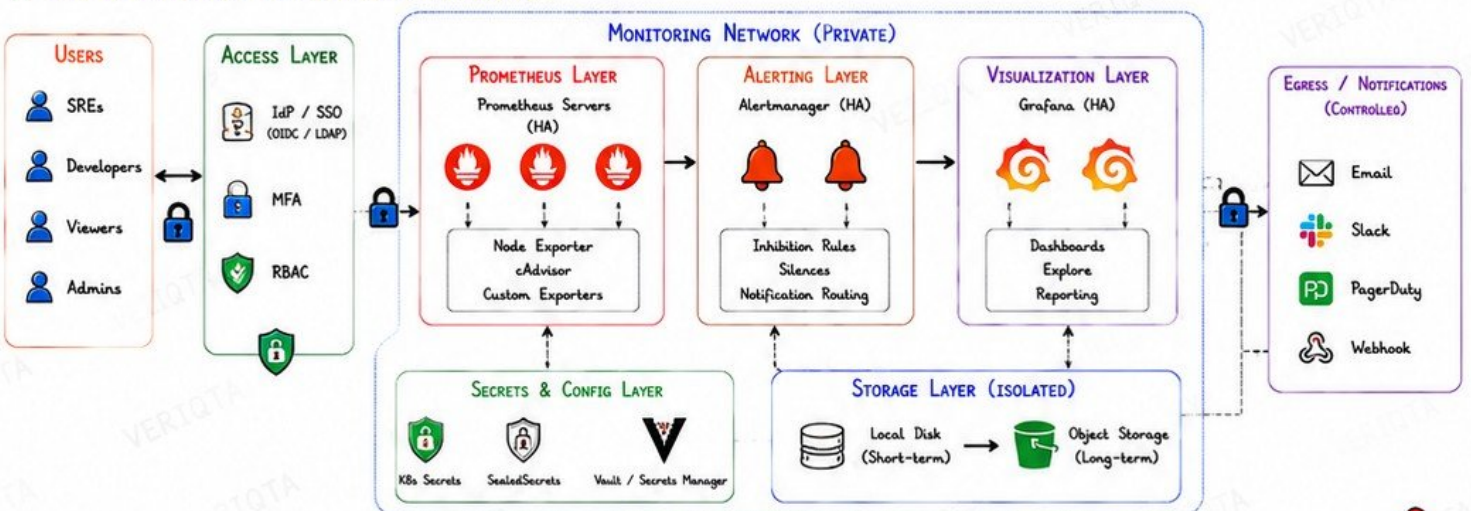
WHAT WE PROTECT

- Metrics Integrity
- Configuration
- Secrets & Keys
- User Access
- Availability

QUICK CHECKLIST

- ✓ Auth enabled
- ✓ TLS enabled
- ✓ RBAC configured
- ✓ Secrets secured
- ✓ Network restricted
- ✓ Logs & audit on
- ✓ Regular reviews

★ SECURE ENTERPRISE MONITORING ARCHITECTURE



SECURITY CONTROLS (FOUNDATION)

- TLS Everywhere (mTLS)
- Firewall & Network Policies
- Audit Logs & Alerting
- Backup & DR Strategy
- Patching & Updates
- Vulnerability Scanning

REMEMBER

A well-secured monitoring system is the backbone of a reliable and trusted observability platform.

6. MULTI-CLUSTER MONITORING

ONE VIEW. ALL CLUSTERS. GLOBAL INSIGHT.

KEY IDEA

Monitor all your Kubernetes clusters from a single pane of glass with consistent metrics, alerts, and dashboards.



1 KUBERNETES CLUSTERS

- Multiple K8s clusters across regions / environments
- Different teams, same visibility
- Consistent monitoring setup
- Labels identify cluster, region, env, team



2 GLOBAL MONITORING

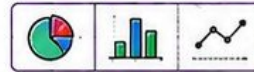
- Centralized long-term storage
- Global query & alerting
- Cross-cluster metrics correlation
- Unified alert routing
- Global SLOs & SLIs

Think Global,
Act Local



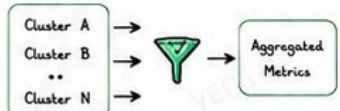
3 CROSS-CLUSTER DASHBOARDS

- Compare clusters side-by-side
- Unified dashboards for apps running everywhere
- Filter by cluster, region, namespace, team
- Drill down from global to local



4 AGGREGATION

- Aggregate metrics across clusters
- Deduplicate series
- Reduce cardinality
- Improve query performance
- Rollup for long-term retention



★ KEY BENEFITS

- ✓ Single pane of glass
- ✓ Global visibility & consistency
- ✓ Faster incident detection
- ✓ Cross-cluster correlation
- ✓ Standardized monitoring
- ✓ Better capacity planning
- ✓ Cost effective with aggregation



CROSS-CLUSTER USE CASES



Incident
Detection

Detect issues early
by correlating
across clusters.



Capacity
Planning

Understand usage
patterns across
all clusters.



SLO / SLI
Tracking

Track global SLOs
for services across
clusters.



Security &
Compliance

Monitor security
posture across
environments.



Cost
Optimization

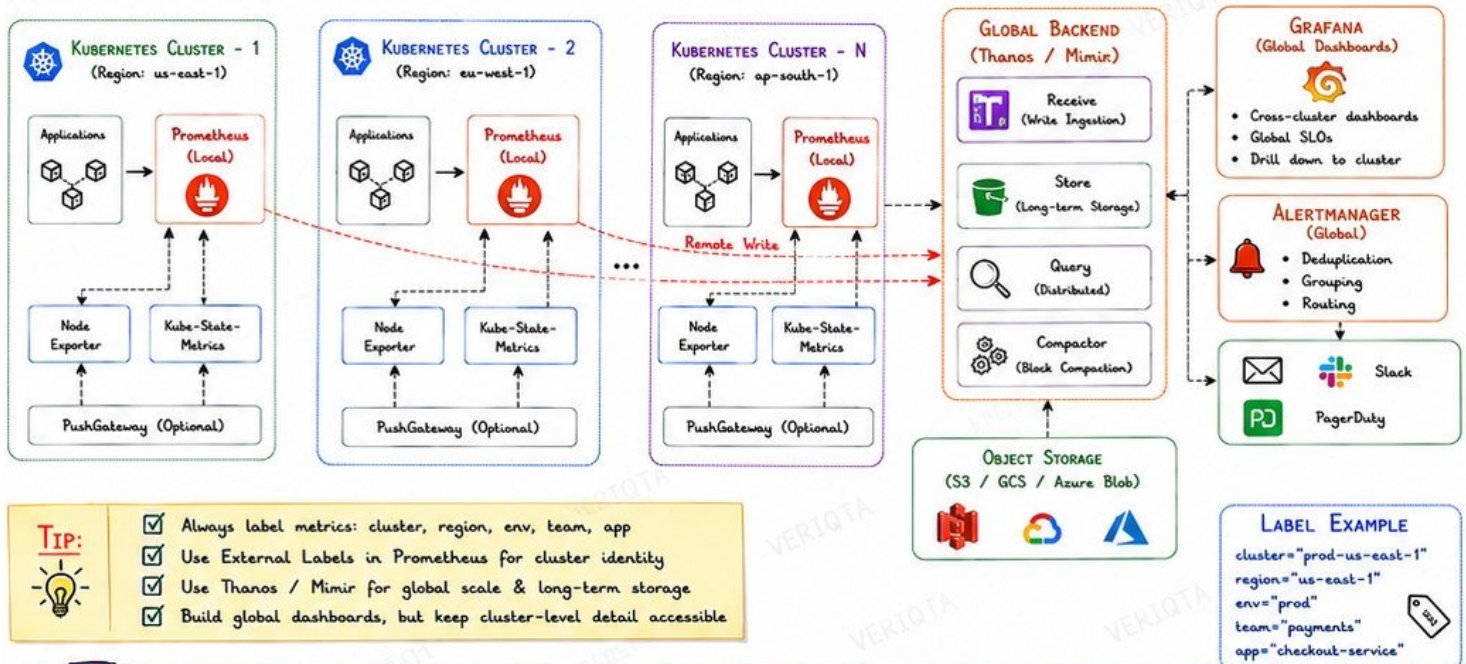
Identify unused
resources across
clusters.

★ BEST PRACTICES

- ✓ Use consistent labels
- ✓ Standardize exporters & configs
- ✓ Label your clusters (region, env, team)
- ✓ Use Thanos / Mimir for scale
- ✓ Keep cardinality under control
- ✓ Automate everything



★ PRODUCTION ARCHITECTURE EXAMPLE



GLOBAL VISIBILITY. LOCAL CONTROL. THAT'S THE POWER OF MULTI-CLUSTER MONITORING.

7. MULTI-CLOUD MONITORING

ONE PLATFORM. ALL CLOUDS. COMPLETE VISIBILITY.

KEY IDEA

Monitor everything across every cloud from a single pane of glass.

1 AWS MONITORING (WITH PROMETHEUS)



- EC2, ECS, EKS, RDS, ELB
- CloudWatch Exporter
- VPC Flow Logs Exporter
- IAM, S3, Lambda metrics
- Route53, API Gateway
- ALB / NLB metrics

Example Targets

- ec2-exporter
- cloudwatch-exporter
- aws-load-balancer-exporter



2 AZURE MONITORING



- VM, AKS, App Service
- Azure Monitor / Insights
- Azure Advisor metrics
- Storage, SQL, Cosmos DB
- Load Balancer, Application GW
- Network & Security metrics

Example Targets

- azure-exporter
- azure-metrics-exporter



3 GOOGLE CLOUD MONITORING



- GCE, GKE, Cloud Run
- Cloud Monitoring metrics
- Stackdriver exporter
- Cloud SQL, BigQuery
- Load Balancers, Pub/Sub
- IAM, VPC, Cloud Functions

Example Targets

- gcp-stackdriver-exporter
- gcp-cloudsql-exporter



4 HYBRID CLOUD



Public Cloud + On-Prem

- Same monitoring for all
- VPN / Direct Connect
- Private Link / Peering
- Consistent metrics model
- Unified alerting & routing

No matter where it runs, it's monitored the same.



5 UNIFIED DASHBOARDS

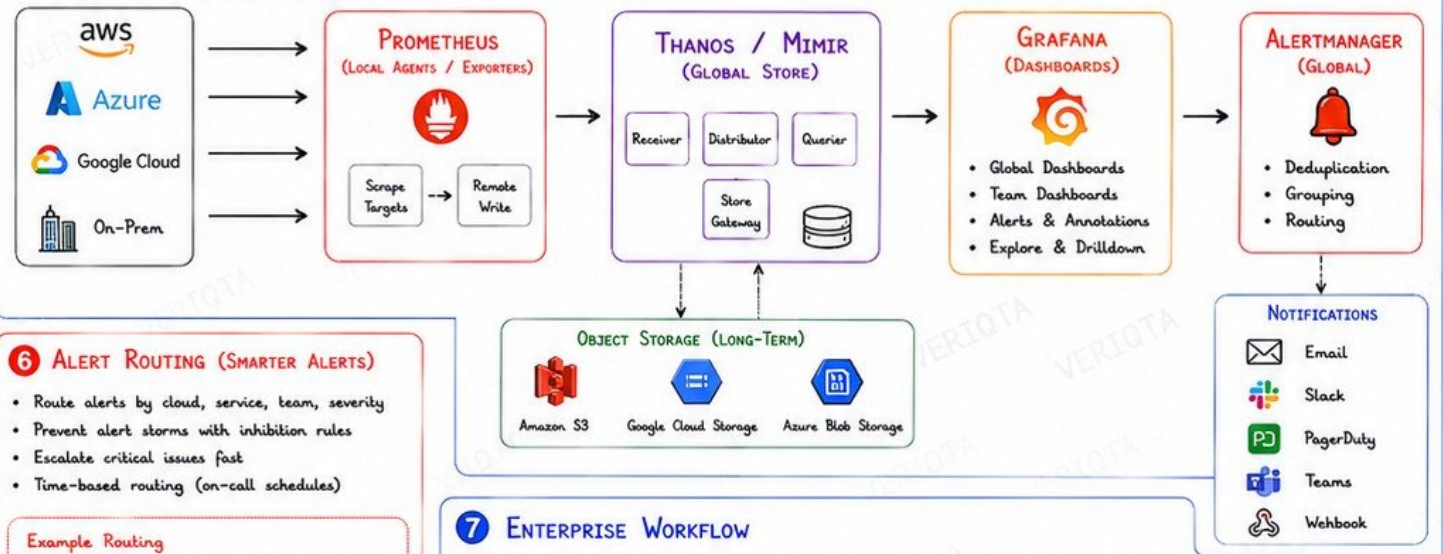


- Global dashboards
- Cross-cloud perspectives
- SLO / SLA tracking
- Cost, Performance, Security
- Drill down to source
- Shared dashboards for teams

ONE PANE OF GLASS for everything.



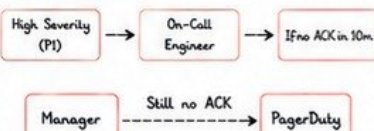
METRICS FLOW (MULTI-CLOUD)



6 ALERT ROUTING (SMARTER ALERTS)

- Route alerts by cloud, service, team, severity
- Prevent alert storms with inhibition rules
- Escalate critical issues fast
- Time-based routing (on-call schedules)

Example Routing



7 ENTERPRISE WORKFLOW



BEST PRACTICES

- ✓ Use consistent labels across all clouds
- ✓ Standardize exporter configurations
- ✓ Use Remote Write for scalability
- ✓ Store long-term data in object storage
- ✓ Create global dashboards & SLOs
- ✓ Test alert routing regularly



CHALLENGES & HOW TO SOLVE

- ✓ Different naming conventions → Normalize labels
- ✓ Network latency → Use regional collectors
- ✓ Cost of storage → Tiered retention, compression
- ✓ High cardinality → Control labels, use recording rules
- ✓ Security → Enforce TLS, RBAC, least-privilege



KEY TAKEAWAY

Multi-cloud monitoring gives you one source of truth across every cloud, every region, every service.



MULTI-CLOUD TODAY. UNIFIED OBSERVABILITY TOMORROW.

8. CAPACITY PLANNING

PLAN TODAY. SCALE CONFIDENTLY. AVOID TOMORROW'S OUTAGE.

KEY IDEA

Capacity planning ensures your monitoring stack can handle current load and future growth without performance issues.

1 FORECASTING GROWTH

- Look at historical trends
- Identify growth patterns
- Understand seasonality
- Plan for spikes & events
- Use data, not guesses

FORECASTING METHODS

- Linear Growth
- Moving Average
- Percentile (P95/P99)
- Exponential Smoothing



2 RESOURCE UTILIZATION OVERVIEW

WATCH THESE METRICS CLOSELY

- ☒ CPU Utilization (%)
- ☒ Memory Utilization (%)
- ☒ Disk I/O & Space
- ☒ Network In/Out
- ☒ Cardinality (Active Series)
- ☒ Scrape Duration
- ☒ Query Duration



GOOD SIGNAL

Stable trends
Predictable growth
Healthy headroom



3 CAPACITY PLANNING PRINCIPLES

- ★ Measure everything
- ★ Understand baseline
- ★ Plan for 2x - 3x growth
- ★ Maintain 20% - 30% headroom
- ★ Test under load
- ★ Review & adjust regularly

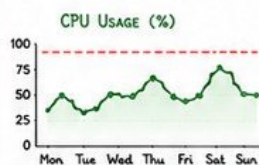
REMEMBER

It's not about today's load.
It's about tomorrow's scale.



4 CPU PLANNING

- Monitor CPU usage
- Track query spikes
- Watch scrape load
- Plan for headroom



TIP
Keep average < 60%
Peak < 80%

5 MEMORY PLANNING

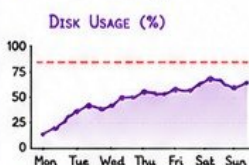
- Monitor memory usage
- Track cardinality growth
- Watch GC pressure
- Plan for 30% headroom



TIP
Avoid OOM kills.
Keep < 70% avg.

6 STORAGE PLANNING

- Monitor disk usage
- Track TSDB growth
- Plan retention wisely
- Use tiered storage



TIP
Use 3-2-1 rule.
Always keep 20% free.

7 NETWORK PLANNING

- Monitor In/Out bandwidth
- Track remote write traffic
- Watch cross-region cost
- Plan for traffic spikes



TIP
Monitor 95th percentile.
Plan for burst.

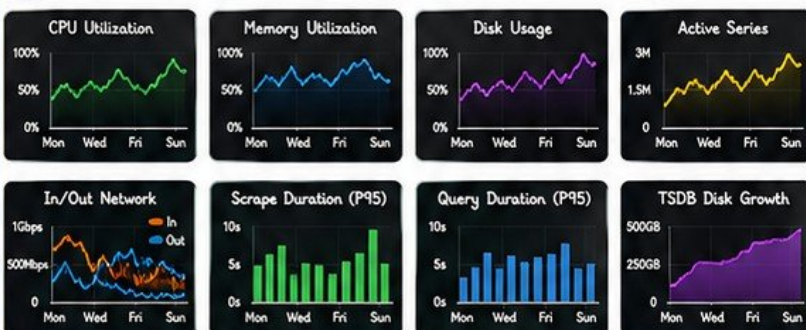
8 KEY CAPACITY METRICS

- Active Series (Cardinality)
- Scrape Duration
- Query Duration (P95)
- WAL Size
- Disk I/O
- Churn Rate
- Exporter Count

THESE METRICS TELL YOU
WHEN TO SCALE.



9 PLANNING DASHBOARDS (WHAT TO TRACK)



DASHBOARD PURPOSE



10 CAPACITY RECOMMENDATION GUIDE

METRIC	WARNING	CRITICAL	ACTION
CPU Usage	> 70%	> 85%	Scale CPU
Memory Usage	> 70%	> 85%	Add Memory
Disk Usage	> 70%	> 85%	Add Storage
Active Series	> 70% limit	> 85% limit	Reduce / Scale
Scrape Duration	> 5s	> 10s	Optimize / Scale
Query Duration (P95)	> 5s	> 10s	Optimize / Upgrade
Network In/Out	> 70% link	> 85% link	Upgrade BW

BEST PRACTICES

- ✓ Review capacity weekly
- ✓ Forecast monthly & quarterly
- ✓ Simulate load before scaling
- ✓ Document assumptions & growth rates
- ✓ Automate alerts on capacity metrics



PRO TIP



Good capacity planning is invisible when it works,
but critical when it's missing.

HEADROOM RULE

Always keep 20% - 30% headroom
for reliability & growth.



REMEMBER



Capacity is not just infra.
It's reliability, performance,
and customer trust.



9. SLI, SLO & SLA

MEASURE RELIABILITY. SET TARGETS. KEEP PROMISES.

KEY IDEA

Reliability is a feature.
SLI defines what you measure,
SLO defines what you promise,
SLA defines what you sign.

1 SLI (SERVICE LEVEL INDICATOR)

A measurable metric that reflects user-perceived reliability.

Examples:

- HTTP request success rate
- API latency (p95)
- Checkout success rate
- Uptime percentage



2 SLO (SERVICE LEVEL OBJECTIVE)

The target value for your SLI over a period of time.

Examples:

- 99.9% availability over 30 days
- 95% of requests < 300ms (p95)
- 99% successful checkouts
- 99.95% uptime



3 SLA (SERVICE LEVEL AGREEMENT)

The formal contract with your customers or stakeholders.

Examples:

- 99.9% uptime monthly
- 4x 9s availability
- Support response time: 1 hour
- Credits if SLO is not met



4 HOW THEY RELATE



SLI feeds the SLO. SLO drives engineering. SLA sets customer expectation.

5 ERROR BUDGETS

The allowed amount of unreliability.

$$\text{Error Budget} = 1 - \text{SLO}$$

Example:

SLO = 99.9% availability
Error Budget = $1 - 0.999 = 0.001 = 0.1\%$

For 30 days (43,200 minutes):

- Allowed downtime = 0.1% of 43,200
= 43.2 minutes



6 ERROR BUDGET STATES

	Budget Healthy (Plenty Left)	Ship features Take risks Move fast
	Budget Caution (Running Low)	Be careful Increase monitoring Fix known issues
	Budget Exhausted (Over Budget)	Stop new features Focus on stability Fix reliability

7 REAL PRODUCTION EXAMPLES

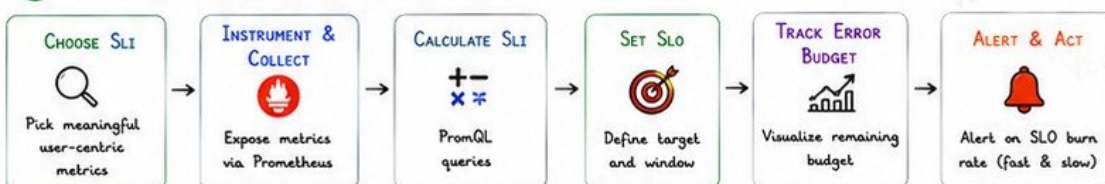
SERVICE	SLI (WHAT WE MEASURE)	SLO (TARGET)	ERROR BUDGET (30 DAYS)	SLA (WE PROMISE)
E-COMMERCE WEBSITE	• HTTP Success Rate (5xx) • Checkout Success Rate	• 99.9% availability • 98% successful checkouts	43.2 minutes (-0.1%)	99.9% Uptime Monthly Credits if below 99.9%
PAYMENT SERVICE	• Payment API Success Rate • p95 Latency	• 99.95% availability • p95 < 200ms	21.6 minutes (-0.05%)	99.95% Uptime Monthly Priority Support
USER AUTH SERVICE	• Login Success Rate • Token Issuance Latency	• 99.99% availability • p95 < 150ms	4.32 minutes (-0.01%)	99.99% Uptime Monthly 24/7 Support
IMAGE UPLOAD SERVICE	• Upload Success Rate • p95 Upload Time	• 99.5% availability • p95 < 500ms	216 minutes (-0.5%)	99.5% Uptime Monthly Best Effort Support

8 INTERVIEW TIPS

- ☒ Always start with SLI. If you can't measure it, you can't improve it.
- ☒ Explain why the metric matters to the user.
- ☒ Show how you calculate the error budget.
- ☒ Explain what happens when the budget is exhausted.
- ☒ Mention visibility, alerting and automation.
- ☒ Tie SLOs to business impact (not just uptime).



9 PRACTICAL IMPLEMENTATION (WITH PROMETHEUS)



GOLDEN RULE

Happy users are the result of good SLOs and healthy error budgets.



RELIABILITY IS A HABIT, NOT A ONE-TIME PROJECT.

Measure it. Protect it. Improve it.



10. RED METHOD

Focus ON THE USER EXPERIENCE.

KEY IDEA

The RED method helps you understand the health of your service from the user's perspective.



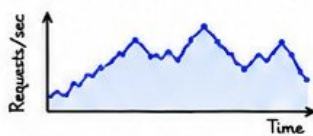
1 REQUESTS



The volume of requests handled by your service.

What it tells you:

- Traffic patterns
- Load on the system
- Capacity needs



Key Metric
request rate (req/sec)

2 ERRORS



The rate of failed requests.

What it tells you:

- Service failures
- Bugs or exceptions
- Dependency issues

Error Rate (%)
 $(\text{errors} / \text{requests}) \times 100$

Key Metric
error rate (%)

3 DURATION



The time taken to serve requests.

What it tells you:

- Performance & latency
- User experience
- Slow dependencies

Track Percentiles
p50, p95, p99, p999

Key Metric
request duration (ms)

4 WHY IT MATTERS



User-Centric

Focuses on what users actually experience.



Simple but Powerful

Works for all services and technologies.



Actionable Insights

Helps you detect, triage, and fix problems faster.

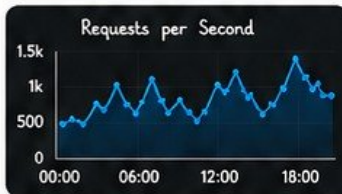


Scalable

Easier to collect, store, and alert at scale.

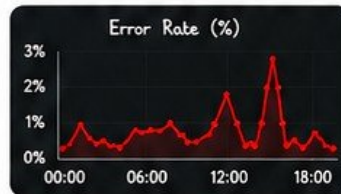
DASHBOARD EXAMPLES (Grafana)

REQUESTS (THROUGHPUT)



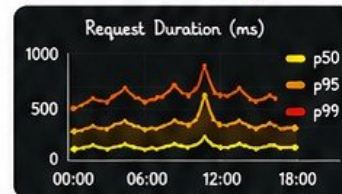
Look for: sudden drops, traffic spikes, unusual patterns

ERRORS (ERROR RATE %)



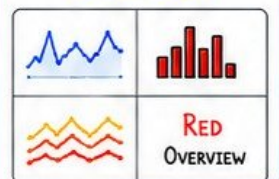
Look for: error spikes, increase above SLO threshold

DURATION (LATENCY)



Look for: high p95/p99, increasing latency trends

GOLDEN SIGNALS IN ONE VIEW



At a glance health of your service



5 PRODUCTION SCENARIOS

SCENARIO	WHAT YOU SEE (RED)	POSSIBLE CAUSE	WHAT YOU DO
Traffic Drop	Requests ↓ Errors normal Duration normal	Upstream issue Outage in dependency DNS / routing issue	Check upstream services Check alerts & logs Validate routing / DNS
Error Spike	Requests normal Errors ↑↑ Duration ↑	Deployment bug Dependency failure Bad input / data	Check logs & traces Rollback if needed Fix & redeploy
High Latency	Requests normal Errors normal Duration (p95/p99) ↑↑	Slow database External API latency Resource saturation	Profile latency Optimize queries Scale resources
Traffic Surge	Requests ↑↑ Errors slightly ↑ Duration ↑	Marketing campaign Viral event Seasonal spike	Auto-scale Increase capacity Tune resources
Partial Outage	Requests ↓↓ Errors ↑↑ Duration very high	Service down Infrastructure failure Cascading failures	Failover / DR Communicate Post-mortem

6 BEST PRACTICES

- ✓ Instrument all critical services
- ✓ Use consistent naming & labels
- ✓ Set SLOs and alert on burn rates
- ✓ Correlate RED with logs & traces
- ✓ Review dashboards with teams
- ✓ Automate alerts and runbooks

TIP

RED tells you that something is wrong. Logs & Traces tell you WHY.



THE POWER OF RED

Users make requests



Your service handles them



RED
METRICS



Monitor, Alert & Analyze



Fix Issues & Improve



Happy Users & Business

11. USE METHOD

Understand INFRASTRUCTURE HEALTH.
COMPLETE VISIBILITY. PROACTIVE ACTION.

KEY IDEA

The **USE** method helps you monitor the resources your system depends on. Perfect for infrastructure and capacity planning.



1 UTILIZATION

How busy a resource is.



What it tells you:

- Usage level of CPU, Memory, Disk, Network
- How much of the resource is being used
- Trend over time

Key Metric Examples

- CPU Utilization (%)
- Memory Utilization (%)
- Disk Usage (%)
- Network Utilization (%)

2 SATURATION

How full a resource is.



What it tells you:

- How close the resource is to its limit
- Predicts future issues
- Critical for capacity planning

Key Metric Examples

- Disk Free Space (%)
- Memory Available (%)
- Inode Usage (%)
- Load Average vs Limit

3 ERRORS

The rate of error events.



What it tells you:

- Hardware errors (I/O, ECC)
- Failed operations
- throttling, drops, timeouts
- Indicates reliability issues

Key Metric Examples

- Disk I/O Errors
- Network Packet Drops
- Failed Mounts
- Kernel / System Errors

4 INFRASTRUCTURE MONITORING USE CASE



- Hosts / Servers
- Virtual Machines
- Kubernetes Nodes
- Storage Systems
- Network Devices

USE = EARLY WARNING OF INFRASTRUCTURE PROBLEMS.

5 PRACTICAL EXAMPLES (PROMETHEUS METRICS)

RESOURCE	UTILIZATION (U)	SATURATION (S)	ERRORS (E)
CPU	$\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5m]) * 100$	$100 - \text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5m])) * 100$	$\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"steal"}\}[5m]) * 100$
Memory	$(1 - \text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes}) * 100$	$\text{node_memory_MemAvailable_bytes}$	$\text{rate}(\text{node_memory_Errors_total}[5m])$
Disk	$\text{rate}(\text{node_disk_io_time_seconds_total}[5m]) * 100$	$\text{node_filesystem_avail_bytes} / \text{node_filesystem_size_bytes} * 100$	$\text{rate}(\text{node_disk_read_errors_total}[5m]) + \text{rate}(\text{node_disk_write_errors_total}[5m])$
Network	$\text{rate}(\text{node_network_transmit_bytes_total}[5m]) / \text{node_network_speed_bytes} * 100$	$\text{node_network_speed_bytes}$	$\text{rate}(\text{node_network_receive_errs_total}[5m]) + \text{rate}(\text{node_network_transmit_errs_total}[5m])$
Load	node_load1	node_cpu_count	N/A

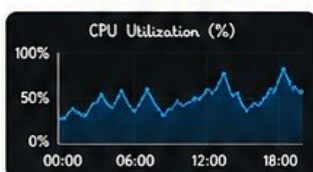
6 USE vs RED

USE (INFRASTRUCTURE)	RED (SERVICES)
System resources	User requests
Capacity & performance focus	Reliability & user experience focus
Proactive (prevent issues)	Reactive (detect issues)
Infrastructure layer	Application / Service layer
Examples: CPU, Memory, Disk, Network	Examples: Requests, Errors, Duration

★ **USE** answers: "Do we have enough resources?"
★ **RED** answers: "How is the service performing?"

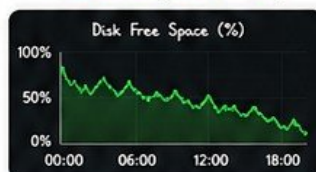
7 DASHBOARD EXAMPLES (GRAFANA)

UTILIZATION (CPU)



Monitor how busy CPU is

SATURATION (DISK FREE %)



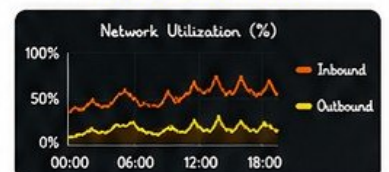
Monitor remaining disk space

ERRORS (I/O ERRORS)



Detect hardware / I/O issues

NETWORK SATURATION



Monitor bandwidth usage

8 PRODUCTION SCENARIOS

High CPU Utilization → CPU close to 100%	Add capacity / optimize processes
Low Disk Saturation → Disk free < 10%	Clean up / expand storage
High Memory Errors → ECC errors increasing	Check hardware / replace faulty RAM
Network Saturation → Bandwidth > 80%	Upgrade link / optimize traffic
I/O Errors Spikes → Disk read/write errors	Check disks / RAID health

9 BEST PRACTICES

- ✓ Monitor USE for all critical resources
- ✓ Set thresholds for saturation metrics
- ✓ Alert before saturation becomes critical
- ✓ Use capacity planning dashboards
- ✓ Correlate USE metrics with RED metrics
- ✓ Review trends weekly
- ✓ Automate scaling where possible



TIP

Combine USE and RED for complete observability:
Healthy infrastructure
→ Happy services
→ Happy users!



USE KEEPS YOUR INFRASTRUCTURE HEALTHY. **RED** KEEPS YOUR USERS HAPPY.



12. THE FOUR GOLDEN SIGNALS

FOCUS ON WHAT MATTERS. IMPROVE WHAT USERS FEEL.

KEY IDEA

Google's SRE approach identifies four metrics that best represent user experience and system health.



1 LATENCY



The time it takes to service a request.

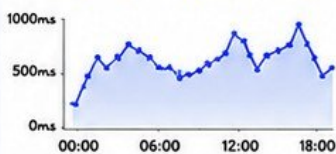
What it tells you:

- User experience (speed)
- Performance of dependencies
- Network or backend issues

Key Metrics

- p50 (median latency)
- p95, p99 (tail latency)
- p99.9 (outliers)

Example (HTTP Request Duration)



2 TRAFFIC



The demand placed on your system.

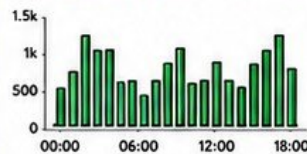
What it tells you:

- How many requests users send
- Growth trends
- Capacity planning needs

Key Metrics

- Requests per second (RPS)
- Throughput
- Incoming connections

Example (Requests Per Second)



3 ERRORS



The rate of requests that result in failures.

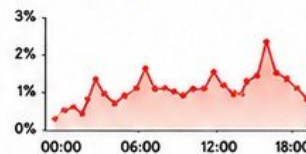
What it tells you:

- Broken features
- Bad deployments
- Dependency or code issues

Key Metrics

- Error rate (%)
- Error count
- Error budget burn

Example (Error Rate %)



4 SATURATION



How full your resources are.

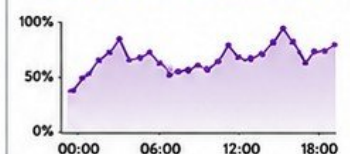
What it tells you:

- Risk of overload
- When to scale
- Capacity headroom

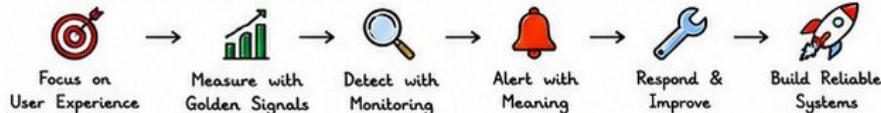
Key Metrics

- CPU Utilization (%)
- Memory Utilization (%)
- Queue Depth, Concurrency

Example (CPU Utilization %)



GOOGLE'S SRE APPROACH



PRINCIPLES

- ✓ Monitor to understand, not just to collect
- ✓ Alert when action is needed
- ✓ Reduce toil with good signals
- ✓ Improve reliability continuously

★ PRODUCTION DASHBOARD EXAMPLE (GRAFANA)



WHY THESE SIGNALS?

- 👤 Directly reflect user experience
- 🔍 Easy to understand and act on
- 📊 Applicable to all services & systems
- 🛡️ Helps achieve SLOs and reliability goals

5 THE FOUR GOLDEN SIGNALS VS RED/USE

ASPECT	FOUR GOLDEN SIGNALS	RED METHOD	USE METHOD
Focus	User experience	Service performance	Infrastructure health
Signals	Latency, Traffic, Errors, Saturation	Requests, Errors, Duration	Utilization, Saturation, Errors
Scope	Service level	Service level	Infrastructure level
Owner	SRE / Dev Teams	SRE / Dev Teams	Ops / Platform Teams
Goal	Happy users	Well-performing services	Healthy infrastructure

6 BEST PRACTICES

- ✓ Define clear SLOs for your services
- ✓ Use percentiles (p95, p99) for latency
- ✓ Correlate signals with logs & traces
- ✓ Set intelligent alerts and runbooks
- ✓ Review dashboards with your team
- ✓ Continuously improve based on insights

TIP

Don't monitor everything. Monitor the right things that impact your users and your business.

13. PERFORMANCE TUNING

TUNE PROMETHEUS for SPEED, SCALE AND STABILITY.

KEY IDEA

Good tuning =
Faster queries,
Lower cost,
Reliable monitoring.

1 SCRAPE OPTIMIZATION

Optimize what you collect.

- Scrape only what matters
- Increase scrape interval for low-value metrics
- Use relabeling to drop unnecessary targets/labels
- Avoid high-cardinality jobs

Example

```
scrape_interval: 30s # default 15s
scrape_timeout: 10s
honor_labels: false
```

2 QUERY OPTIMIZATION

Write efficient queries.

- ✓ Use `rate()` over `increase()`
- ✓ Filter early, aggregate late
- ✓ Avoid `.*` regex on labels
- ✓ Use recording rules for expensive queries
- ✓ Use subqueries for rolling calculations

Example (Good)

```
rate(http_requests_total[5m])
by (job)
```

3 RECORDING RULES

Pre-compute expensive queries.

- Speeds up dashboards
- Reduces query load
- More consistent results
- Enables alert performance

Example (rules.yml)

```
groups:
- name: job:http
  interval: 30s
  rules:
- record: job:http:request_rate5m
  expr: rate(http_requests_total[5m])
```

4 LABEL OPTIMIZATION

Control cardinality.

- Remove unneeded labels
- Avoid dynamic labels (`user_id`, `session_id`)
- Use `relabel_configs` to drop or rewrite labels
- Monitor cardinality continuously

Example (relabel)

```
relabel_configs:
- action: labeldrop
  regex: "(pod_ip|container_id)"
```

5 TSDB TUNING

Tune the storage engine.

- Adjust block duration
- Configure retention
- Use WAL compression
- Tune compaction concurrency

Example (prometheus.yml)

```
storage:
  tsdb:
    retention.time: 30d
    retention.size: 50GB
    min-block-duration: 2h
    max-block-duration: 2h
```

6 MEMORY OPTIMIZATION

Keep Prometheus healthy.

- Set proper memory limits
- Tune head chunk size
- Enable mmap
- Reduce label cardinality
- Monitor `/metrics` for memory usage

Useful Flags

```
--storage.tsdb.wal-compression
--storage.tsdb.mmap
--query.max-concurrency=20
--query.timeout=2m
```

PERFORMANCE TUNING CHECKLIST

- ✓ Scrape only what matters
- ✓ Use efficient PromQL queries
- ✓ Create recording rules for heavy queries
- ✓ Keep label cardinality in check
- ✓ Configure TSDB retention & compaction
- ✓ Monitor memory and resource usage

PROMETHEUS PERFORMANCE ARCHITECTURE

Targets



Scrape
Optimization

Prometheus Server

- ✓ Efficient Ingestion
- ✓ Optimized Storage (TSDB)
- ✓ Fast Query Engine

Recording Rules
(Pre-compute)

Grafana
Dashboards
& Alerts

TSDB (Optimized Storage)

BEFORE VS AFTER TUNING

BEFORE TUNING

- ✗ High CPU & Memory usage
- ✗ Slow dashboards (>10s)
- ✗ Query timeouts
- ✗ High cardinality (millions of series)
- ✗ Large storage usage

AFTER TUNING

- ✓ Lower CPU & Memory usage
- ✓ Fast dashboards (<2s)
- ✓ No query timeouts
- ✓ Low cardinality (optimized)
- ✓ Efficient storage usage

BEST PRACTICES

- ✓ Understand your metrics and business needs
- ✓ Collect less, do more with strategic metrics
- ✓ Use recording rules for dashboards & alerts
- ✓ Keep queries simple and efficient
- ✓ Regularly review cardinality and retention
- ✓ Monitor Prometheus itself (self-monitoring)

TIP.

The best performance improvement is often collecting the **RIGHT** metrics – not more metrics.

TUNING IMPACT SUMMARY

Tuning Area	Improves	Reduces	Metric to Watch
Scrape Optimization	Ingestion speed	CPU, Network, Series	<code>scrape_duration_seconds</code>
Query Optimization	Query performance	Latency, CPU	<code>prometheus_engine_query_duration_seconds</code>
Recording Rules	Dashboard speed	Query load, Latency	<code>prometheus_rule_evaluation_duration_seconds</code>
Label Optimization	Storage & Memory	Cardinality, Costs	<code>prometheus_tsdb_head_series</code>
TSDB Tuning	Storage efficiency	Disk I/O, Compaction time	<code>prometheus_tsdb_compactions_total_time</code>
Memory Optimization	Stability & Reliability	OOM, GC pressure	<code>process_resident_memory_bytes</code>

MONITOR PROMETHEUS ITSELF

- CPU Usage < 70%
- Memory Usage < 75%
- Head Series stable
- WAL Size growing slowly
- Query Duration p95 < 2s
- Rule Evaluation p95 < 10s

Use `/targets`, `/tsdb-status`, `/metrics` and Grafana dashboards.

14. DISASTER RECOVERY

BE PREPARED. RECOVER FAST. MINIMIZE IMPACT.

KEY IDEA

A good monitoring system should survive disasters and help you recover quickly with minimal data loss.



1 BACKUP STRATEGY

Protect your monitoring data and configuration.

- Regular snapshots
- Retention policies
- Configuration backup
- Encryption at rest

Example

- Snapshot TSDB every 4h
- Retain 15 days locally
- Backup to S3 daily
- Version control for configs (Git)

2 RESTORE PROCEDURES

Document and automate restore steps.

- Step-by-step runbook
- Automated restore scripts
- Validate data integrity
- Minimize RTO/RPO

Example

1. Stop Prometheus
2. Restore TSDB snapshot
3. Restore configuration
4. Start Prometheus
5. Validate & alert

3 REMOTE STORAGE

Store backups off-site safely.

- Object storage (S3/GCS)
- Cross-region replication
- Lifecycle policies
- Encrypted backups

Example

- Upload to S3 (us-east-1)
- Replicate to eu-west-1
- Glacier after 30 days
- SSE-KMS encryption

4 MULTI-REGION RECOVERY

Run or replicate across regions.

- Active-Passive setup
- Data replication
- DNS failover
- Global dashboards

Example

- Primary: us-east-1
- DR: eu-west-1
- DNS failover (Route 53)
- RPO: < 15 min
- RTO: < 60 min

5 TESTING RECOVERY PLANS

Test regularly. Don't wait for disaster.

- Scheduled DR drills
- Validate backups
- Simulate failures
- Track RTO/RPO

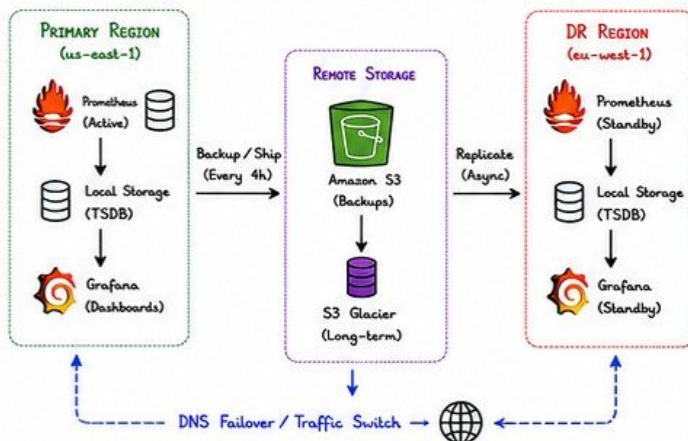
Example

- DR drill every quarter
- Restore from backup
- Measure RTO/RPO
- Update runbooks

6 PRODUCTION CHECKLIST

- ☒ Backups automated
- ☒ Off-site storage
- ☒ Restore tested
- ☒ Runbooks ready
- ☒ Alerts configured
- ☒ DR plan documented
- ☒ Team trained

DISASTER RECOVERY ARCHITECTURE



DISASTER RECOVERY RUNBOOK (HIGH LEVEL)

1. **Detect Incident**
Identify outage or major failure.
2. **Assess Impact**
Determine affected components & data.
3. **Activate DR**
Switch to DR region or restore from backup.
4. **Restore Data**
Restore latest TSDB snapshot & configs.
5. **Validate & Test**
Check data, alerts, dashboards & queries.
6. **Communicate**
Notify team & stakeholders.
7. **Post-Recovery Review**
Review timeline & improve the plan.

RTO & RPO TARGETS

- RTO (Recovery Time Objective)
< 60 minutes
- RPO (Recovery Point Objective)
< 15 minutes

GOOD TOOLS

- Prometheus + Thanos
- AWS S3 + Glacier
- Velero (for K8s)
- Route 53 (DNS Failover)
- Terraform (Infra as Code)
- PagerDuty / Opsgenie

★ PRODUCTION DASHBOARD EXAMPLE (GRAFANA)



Key Alerts

- Backup failed
- Backup too old
- RPO above threshold
- Storage almost full

Alert Example (Prometheus)

```
ALERT BackupTooOld
IF time() - backup_last_success_timestamp > 4h
FOR 10m
LABELS { severity="critical" }
ANNOTATIONS { summary="Backup is older than 4h" }
```

BACKUP STRATEGY TABLE

COMPONENT	WHAT TO BACKUP	FREQUENCY	RETENTION	LOCATION
Prometheus (TSDB)	TSDB snapshots	Every 4h	15 days	S3 (us-east-1) + Glacier
Configuration	prometheus.yml, rules, alerts	Daily	30 days	S3
Grafana	Dashboards, Data Sources	Daily	30 days	S3
Alertmanager	Config & Senders	Daily	30 days	S3
Kubernetes	Persistent Volumes (if any)	Daily	15 days	S3 / Velero



FINAL DISASTER RECOVERY CHECKLIST

- ☒ Backups are automated and monitored
- ☒ Backups stored in remote/off-site location
- ☒ RPO and RTO targets are defined
- ☒ Restore procedures documented
- ☒ DR runbooks are up to date
- ☒ Regular DR drills are performed
- ☒ Backup integrity is validated
- ☒ Multi-region setup tested
- ☒ Team trained on DR procedures
- ☒ Critical alerts for DR are configured
- ☒ Dependencies (DNS, IAM, Network) ready
- ☒ Post-incident review process defined

TIP

Test your DR plan before you need it. The best time to find problems is not during a disaster!



“

Disaster will test your plan. Preparation will protect your business.



15. PRODUCTION CASE STUDIES

REAL INCIDENTS. REAL IMPACT. REAL LESSONS.

KEY IDEA

Learn from production incidents so you can prevent them in your environment.



1 KUBERNETES OUTAGE



WHAT HAPPENED

A bad rolling update caused multiple pods to crash, leading to service unavailability.

IMPACT

100% downtime for 23 minutes for checkout service.

ROOT CAUSE

Liveness probe misconfigured & wrong maxUnavailable setting.

HOW WE FIXED IT

- ✓ Rollback deployment
- ✓ Fixed liveness probe
- ✓ Set maxUnavailable: 0
- ✓ Added canary + alerts

LESSON LEARNED

Always test updates and use safe rollout strategies.

2 EXPORTER FAILURE



WHAT HAPPENED

Node Exporter stopped running on 40% of nodes after OS upgrade.

IMPACT

Missing metrics, blind spots in dashboards & alerts.

ROOT CAUSE

Systemd service not enabled after upgrade.

HOW WE FIXED IT

- ✓ Reinstalled & enabled service
- ✓ Created DaemonSet
- ✓ Added self-monitoring alert
- ✓ Verified after upgrades

LESSON LEARNED

Monitor your monitoring with alerts.

3 HIGH CARDINALITY INCIDENT



WHAT HAPPENED

A new label (user_id) on a metric caused millions of time series.

IMPACT

Prometheus OOMKilled, queries slow, alerts delayed.

ROOT CAUSE

Unbounded label value (user_id) in metric.

HOW WE FIXED IT

- ✓ Removed high-cardinality label
- ✓ Applied relabeling rules
- ✓ Set cardinality budgets
- ✓ Added recording rules

LESSON LEARNED

Control cardinality or it will control you.

4 STORAGE EXHAUSTION



WHAT HAPPENED

TSDB storage filled up to 100%, Prometheus stopped ingesting data.

IMPACT

Data loss for 2 hours, alerts missing.

ROOT CAUSE

Retention too high, no storage monitoring.

HOW WE FIXED IT

- ✓ Increased disk space
- ✓ Reduced retention
- ✓ Enabled compaction tuning
- ✓ Added storage alerts

LESSON LEARNED

Right-size storage and monitor it always.

5 ALERT STORM



WHAT HAPPENED

A network glitch triggered thousands of alerts across multiple services.

IMPACT

On-call overwhelmed, missed real alerts.

ROOT CAUSE

No alert grouping, no inhibition, noisy alerts.

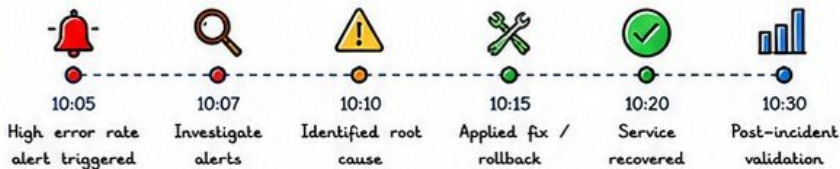
HOW WE FIXED IT

- ✓ Added alert grouping
- ✓ Configured inhibition rules
- ✓ Adjusted thresholds
- ✓ Added alert rate limits

LESSON LEARNED

Reduce noise so real alerts are never missed.

EXAMPLE INCIDENT TIMELINE (REALISTIC)



★ Fast detection + clear dashboards + good runbooks = faster recovery

KEY LESSONS FROM ALL INCIDENTS



- ✓ Monitoring is only useful when it is reliable.
- ✓ Alert only on actionable issues.
- ✓ High cardinality is a silent killer.
- ✓ Storage, retention & capacity must be monitored.
- ✓ Test your alerts, dashboards & runbooks.
- ✓ Automate boring tasks, document everything.
- ✓ Blameless postmortems drive real improvements.

IMPACT SUMMARY AT A GLANCE

	INCIDENT	DURATION	USER IMPACT	ROOT CAUSE	SEVERITY	RECOVERED IN
1	Kubernetes Outage	23 min	100%	Bad rollout + probes	SEV-1	18 min
2	Exporter Failure	1h 12m	Partial (missing metrics)	Service not enabled	SEV-2	45 min
3	High Cardinality	2h 05m	Degraded (slow + missing)	Unbounded labels	SEV-1	1h 20m
4	Storage Exhaustion	2h 00m	Data loss (2 hours)	Retention too high	SEV-1	35 min
5	Alert Storm	30 min	On-call overwhelmed	No grouping / inhibition	SEV-2	25 min

WHAT GOOD WOULD HAVE PREVENTED THESE?



Change Management
Test changes in staging, use canaries.



Monitoring of Monitoring
Alert if exporters or Prometheus are down.



Cardinality Control
Review labels, set budgets, use relabeling.



Capacity Planning
Monitor storage, set retention & alerts.



Alert Hygiene
Group, inhibit, rate-limit, use clear severity.



Runbooks & Drills
Document steps and practice regularly.

TIP



Every incident is a teacher.
The goal is not to avoid all failures, but to fail better and recover faster.

POST-INCIDENT CHECKLIST

- ✓ What happened?
- ✓ Why did it happen?
- ✓ What was the impact?
- ✓ What actions fixed it?
- ✓ How can we prevent?
- ✓ Update runbooks
- ✓ Share learnings
- ✓ Track improvements

“

Great systems are built not by avoiding failures, but by learning from them.
— SRE Mindset



16. DEBUGGING REAL INCIDENTS

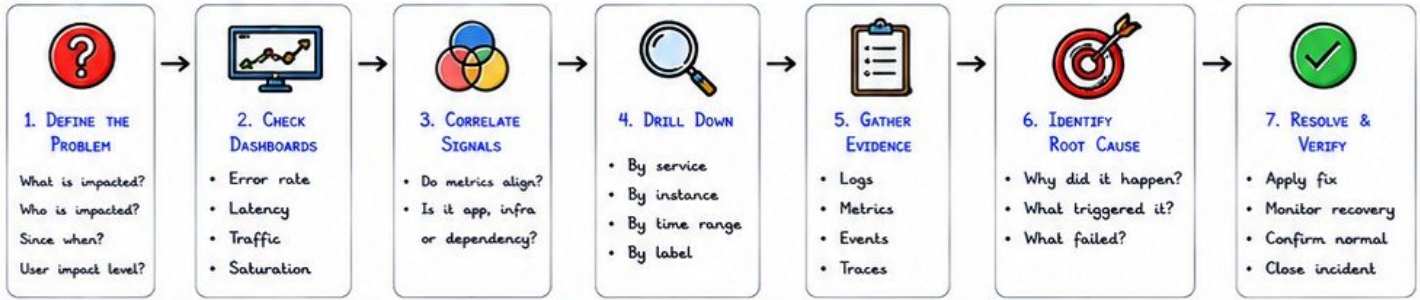
OBSERVE. INVESTIGATE. ISOLATE. FIX. VERIFY.

KEY IDEA

A structured approach helps you find the real root cause — not just the noisy symptom.



1 STEP-BY-STEP INVESTIGATION WORKFLOW



2 LOGS - FIND THE STORY



What to look for:

- Errors & stack traces
- Exceptions & timeouts
- Unusual spikes
- Failed external calls

Example (Loki Query)

```
{job="api-server"} |!= "error" | unwrap time | line_format "{.}"
```

Use filters, levels, and time range to narrow quickly.

3 METRICS - FIND THE PATTERN



What to look for:

- Sudden spikes or drops
- Resource saturation
- Error rate increase
- High latency
- Traffic anomalies

Example (PromQL)

```
rate(http_requests_total{job="api" status=~"5.."}[5m])
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, job))
```

4 PROMQL - POWERFUL INVESTIGATION



ESSENTIAL PROMQL FOR DEBUGGING

- Error Rate**
`rate(http_requests_total{status=~"5.."}[5m])`
- High Latency (p95)**
`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, job))`
- CPU Usage by Pod**
`sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)`
- Memory Usage by Pod**
`sum(container_memory_working_set_bytes) by (pod)`
- Top Failing Endpoints**
`topk(10, sum(rate(http_requests_total{status=~"5.."}[5m])) by (path))`

PROMQL TIPS

- Start simple
- Use time range
- Compare before/after
- Use by() for breakdown
- Use topk() to find the worst offenders
- Validate with multiple signals

5 ROOT CAUSE ANALYSIS (RCA)



Common Techniques

- 5 Whys
- Isolate variables
- Follow the request flow
- Check recent changes
- Correlate logs + metrics
- Check dependencies
- Check resource limits
- Check external services

Example RCA

```
High 5xx errors
↓
API timeouts
↓
DB connection pool exhausted
↓
Slow queries
↓
Missing index after deploy
↓
Root Cause: Missing DB index
```

6 RESOLUTION WORKFLOW



8 INCIDENT INVESTIGATION CHECKLIST



- Understand the problem & impact
- Check dashboards (Golden Signals)
- Check alerts & recent changes
- Check logs for errors
- Query metrics with PromQL
- Drill down by service/instance
- Check dependencies & external services
- Identify the root cause
- Apply fix and monitor
- Verify recovery
- Communicate updates
- Document & improve

TIP:



Trust data, not guesses.
Correlate multiple signals before acting.
Slow down to speed up!

“The best engineers aren't the ones who never see incidents, it's the ones who debug smart, learn fast, and make it better.”



17. SENIOR ENGINEER INTERVIEW SCENARIOS

NOT JUST ANSWERS. SHOW THINKING. EXPLAIN TRADE-OFFS. OWN THE OUTCOME.

KEY IDEA

Interviewers want to see how you think, not just what you know.



1 ARCHITECTURE QUESTIONS



Example Questions

- How would you design a monitoring solution for a multi-region system?
- How does Prometheus achieve scalability?
- What are the components of Prometheus architecture?
- How would you monitor 10,000 services?
- How would you ensure high availability for Prometheus?

What Interviewers Look For

- ✓ System design ability
- ✓ Understanding of components
- ✓ Scalability & HA considerations
- ✓ Trade-off analysis

2 TROUBLESHOOTING QUESTIONS



Example Questions

- Prometheus is not scraping targets. How do you debug?
- High cardinality is causing memory issues. What do you do?
- Alerts are firing incorrectly. How do you investigate?
- Grafana dashboards are slow. What's your approach?
- Missing metrics after deployment. How do you troubleshoot?

What Interviewers Look For

- ✓ Problem-solving approach
- ✓ Use of tools & commands
- ✓ Depth of debugging
- ✓ Root cause mindset

3 DESIGN QUESTIONS



Example Questions

- Design a monitoring solution for a Kubernetes platform.
- Design alerting for a payment system (high availability).
- Design a system to handle millions of metrics.
- How would you design dashboards for SRE teams?
- Design a Prometheus setup for compliance & audit.

What Interviewers Look For

- ✓ Structured design thinking
- ✓ Understanding of requirements
- ✓ Trade-offs & justifications
- ✓ Operational excellence

4 EXPECTED ANSWER FRAMEWORK



Understand the Problem
Ask clarifying questions. Define requirements.



Break it Down
Identify components, constraints, and goals.



Propose Options
Discuss multiple approaches with pros & cons.



Deep Dive
Explain the chosen approach in detail.



Consider Trade-offs
Scalability, cost, reliability, operability, security.



Operational Thinking
Monitoring, alerting, failure scenarios, runbooks.

SENIOR ENGINEER INTERVIEW SCENARIOS

SCENARIO	TYPE	QUESTION	EXPECTED ANSWER (HIGH LEVEL)	WHAT INTERVIEWERS LOOK FOR
1 Multi-Region Monitoring Architecture	Architecture	Design a monitoring solution for a multi-region microservices platform.	• Use regional Prometheus servers • Thanos/Cortex for long-term storage & global query • Push alerts to Alertmanager (HA) • Grafana with global dashboards • Consider latency, data residency & HA	• System design • Scalability • Trade-offs • Operational awareness
2 Prometheus Not Scraping Targets	Troubleshooting	Targets are down in targets page. How do you debug?	• Check target config & service discovery • Verify network (ping, telnet, DNS) • Check /targets, /targets?state=active • Check scrape metrics & errors • Review firewall, auth, TLS, relabeling	• Methodical approach • Use of right tools • Root cause analysis • Attention to detail
3 High Cardinality Incident	Troubleshooting	High cardinality is causing memory issues. What do you do?	• Identify offending labels (topk, count by) • Use labeldrop/keep, relabeling • Set cardinality budgets & alerts • Review metric naming & instrumentation • Consider recording rules	• Metrics knowledge • Impact understanding • Preventive mindset • Practical solutions
4 Design Alerting for Payment System	Design	Design alerting for a payment system.	• Define SLIs & error budgets • Multi-level alerts (service, dependency, infra) • Alert routing, deduplication, inhibition • On-call schedules & runbooks • Noise reduction & actionable alerts	• SLI/SLO knowledge • Alerting best practices • Reliability mindset • Operational maturity
5 Dashboard Performance Issue	Troubleshooting	Grafana dashboards are slow. What's your approach?	• Check datasource & query inspector • Optimize PromQL (reduce range, use rate) • Use recording rules & pre-aggregations • Check cardinality & panel count • Enable caching & tune intervals	• Query optimization • Performance tuning • Tool proficiency • User experience focus

WHAT INTERVIEWERS REALLY WANT TO SEE



Problem Solving



Structured Thinking



Trade-off Analysis



Deep Technical Knowledge



Operational Mindset



Clear Communication

TIPS TO CRUSH THE INTERVIEW

- ✓ Think out loud. Explain every step.
- ✓ Ask clarifying questions.
- ✓ Use real-world examples.
- ✓ Discuss trade-offs, not just solutions.
- ✓ Show ownership and follow-through.



REMEMBER

“Senior engineers don't just solve problems, they prevent them, design for scale, and make systems better.”



18. MONITORING SYSTEM DESIGN EXERCISES

DESIGN IT. MONITOR IT. TRUST IT.

KEY IDEA

A great monitoring system is built with scalability, reliability and actionable insights in mind.



1 DESIGN A STARTUP MONITORING PLATFORM



Requirements

- 10 microservices
- 100 RPS
- Single region (AWS)
- Team of 5 engineers
- Cost sensitive

What to Include

- ✓ Metrics collection
- ✓ Alerting
- ✓ Dashboards
- ✓ Logging (basic)
- ✓ Cost optimization

2 DESIGN ENTERPRISE MONITORING PLATFORM



Requirements

- 500+ microservices
- 10K+ RPS
- Multiple teams
- Compliance (SOC2)
- 99.99% availability

What to Include

- ✓ Scalable metrics pipeline
- ✓ Multi-tenancy
- ✓ RBAC & security
- ✓ Global dashboards
- ✓ Audit & retention

3 DESIGN MULTI-REGION MONITORING



Requirements

- 3 regions (us-east-1, eu-west-1, ap-south-1)
- Cross-region services
- Global SLOs
- Low latency dashboards

What to Include

- ✓ Regional isolation
- ✓ Global aggregation
- ✓ Replication strategy
- ✓ Cross-region alerting
- ✓ Disaster recovery

4 DESIGN HIGH AVAILABILITY MONITORING STACK



Requirements

- Zero data loss (or minimal)
- Survive AZ/Region failure
- High ingestion rate
- Fast query performance

What to Include

- ✓ HA Prometheus
- ✓ Long-term storage
- ✓ Alertmanager HA
- ✓ Grafana HA
- ✓ Backup & recovery

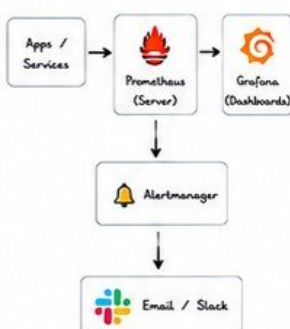
DESIGN PROCESS

- 1 Understand Requirements
- 2 Identify Metrics & Data Sources
- 3 Choose Architecture Components
- 4 Consider Scalability & HA
- 5 Define Alerting & Dashboards
- 6 Security & Access Control
- 7 Cost & Operational Efficiency

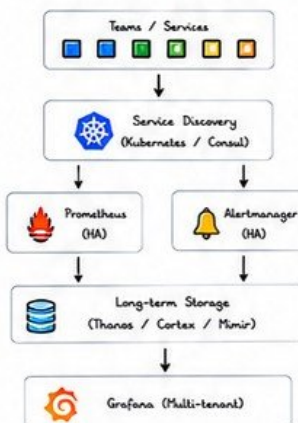


REFERENCE ARCHITECTURE PATTERNS

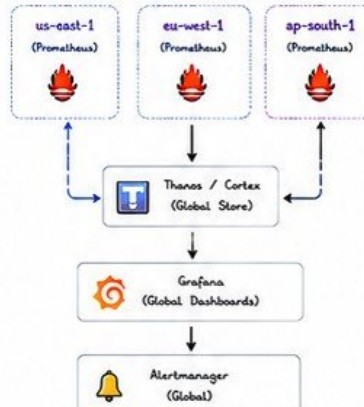
A. STARTUP (SINGLE REGION)



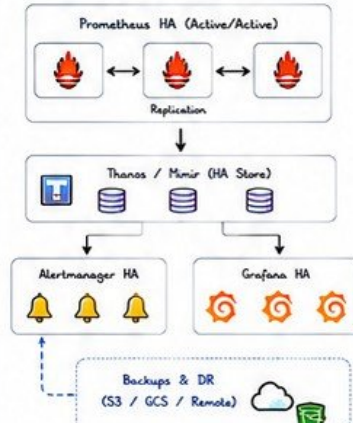
B. ENTERPRISE (MULTI-TEAM)



C. MULTI-REGION



D. HIGH AVAILABILITY



SOLUTION WALKTHROUGHS (EXAMPLE)

1 Startup Platform

- Use single Prometheus with 2h retention on SSD
- Node Exporter, cAdvisor, Blackbox Exporter
- Alertmanager with email + Slack
- Grafana with prebuilt dashboards
- Weekly snapshot to S3

Why it works

- ✓ Simple to operate
- ✓ Low cost
- ✓ Easy to scale later

2 Enterprise Platform

- HA Prometheus with Thanos for long-term storage
- Centralized Alertmanager (HA)
- Team-based dashboards with RBAC
- Compliance metrics & audit logs
- Integration with PagerDuty

Why it works

- ✓ Scalable
- ✓ Secure & compliant
- ✓ Team isolation

3 Multi-Region Platform

- Regional Prometheus clusters
- Thanos Sidecar + Store Gateway in each region
- Global Query Frontend for unified view
- Global Alertmanager with routing rules

Why it works

- ✓ Low latency locally
- ✓ Global visibility
- ✓ Resilient to region loss

4 High Availability Platform

- Prometheus HA pair per region
- Thanos/Mimir with object storage replication
- Alertmanager cluster with gossip
- Grafana behind load balancer

Why it works

- ✓ High availability
- ✓ No single point of failure
- ✓ Data durability

KEY METRICS TO CONSIDER

- ✓ Service: `rate(http_requests_total[5m])`
- ✓ Errors: `rate(errors_total[5m])`
- ✓ Latency: `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))`
- ✓ Saturation: `rate(container_cpu_usage_seconds_total[5m]) / kube_pod_container_resource_limits_cpu_cores`
- ✓ Availability: `up == 1`
- ✓ Traffic: `rate(http_requests_total[5m])`

BEST PRACTICES

- ★ Start simple, iterate and improve.
- ★ Monitor the monitoring system itself.
- ★ Use RED / USE & Golden Signals.
- ★ Keep cardinality under control.
- ★ Design for failure, test recovery.
- ★ Automate onboarding of new services.



TIP

The best monitoring design is the one your team will maintain and rely on every day.

19. PRODUCTION READINESS CHECKLIST

BUILD IT RIGHT. MONITOR IT RIGHT. SLEEP AT NIGHT.

KEY IDEA

Use this checklist before going live to ensure your monitoring system is production ready.



1 INFRASTRUCTURE



- ✓ Highly available Prometheus (HA setup)
- ✓ Persistent storage configured
- ✓ Sufficient CPU, RAM, and Disk IOPS
- ✓ Network latency between nodes is low
- ✓ Time sync (NTP) enabled
- ✓ Backup & retention strategy in place
- ✓ Capacity planning completed

2 SECURITY



- ✓ TLS enabled for all components
- ✓ Authentication enabled (Basic Auth / OIDC)
- ✓ RBAC configured properly
- ✓ Network policies / firewall rules applied
- ✓ Secrets stored securely (Vault / KMS)
- ✓ Least privilege access principle enforced
- ✓ Audit logging enabled

3 MONITORING COVERAGE



- ✓ All critical services instrumented
- ✓ Blackbox monitoring for endpoints
- ✓ Infrastructure metrics covered (CPU, RAM, Disk, Network)
- ✓ Business metrics covered (if applicable)
- ✓ Use RED / USE / Golden Signals
- ✓ No critical gaps in monitoring
- ✓ Service discovery working

4 DASHBOARDS



- ✓ Executive overview dashboard created
- ✓ Service-level dashboards available
- ✓ Golden Signals dashboard available
- ✓ Capacity & saturation dashboard
- ✓ Infrastructure dashboard
- ✓ Dashboards are clear, actionable & tested
- ✓ Variable & templating working

5 ALERTS



- ✓ Critical alerts defined & configured
- ✓ Alert routing rules configured
- ✓ Escalation policies configured
- ✓ Alert thresholds tested
- ✓ No noisy / duplicate alerts
- ✓ Silences & maintenance windows configured
- ✓ On-call notifications tested

6 DOCUMENTATION



- ✓ Architecture diagram up to date
- ✓ Monitoring stack documentation
- ✓ Component inventory
- ✓ Data retention policy documented
- ✓ Alerting policy documented
- ✓ Access information documented
- ✓ Change log maintained

7 RUNBOOKS



- ✓ Incident response runbook available
- ✓ Common alerts runbooks written
- ✓ Escalation matrix available
- ✓ Troubleshooting guides available
- ✓ Disaster recovery runbook
- ✓ On-call handover procedure documented
- ✓ Regular runbook review

8 VALIDATION CHECKLIST



- ✓ Failover test completed
- ✓ Backup & restore test completed
- ✓ Alert test (end-to-end) completed
- ✓ Dashboard validation completed
- ✓ Load test / capacity test completed
- ✓ Security test completed
- ✓ Monitoring system approved for production

PRODUCTION GO-LIVE CHECKLIST



- ✓ All above sections are complete
- ✓ Stakeholders reviewed & approved
- ✓ Monitoring team trained
- ✓ Runbooks & contacts shared
- ✓ Communication plan ready
- ✓ Go/No-Go decision taken

“

Good monitoring is not just about collecting metrics, it's about being ready when things go wrong.

FINAL REMINDER



- ✓ Monitor your monitoring.
- ✓ Continuously improve.
- ✓ Keep it simple.
- ✓ Automate everything you can.
- ✓ Document what you build.
- ✓ Review regularly.



TIP:

Re-run this checklist before every major release or infrastructure change. Production readiness is a habit, not a one-time task!

✓ = Done ⌚ = In Progress ✗ = Not Done ? = N/A

20. FINAL ENTERPRISE MONITORING LAB

HANDS-ON CAPSTONE PROJECT

BUILD A COMPLETE ENTERPRISE MONITORING PLATFORM USING:

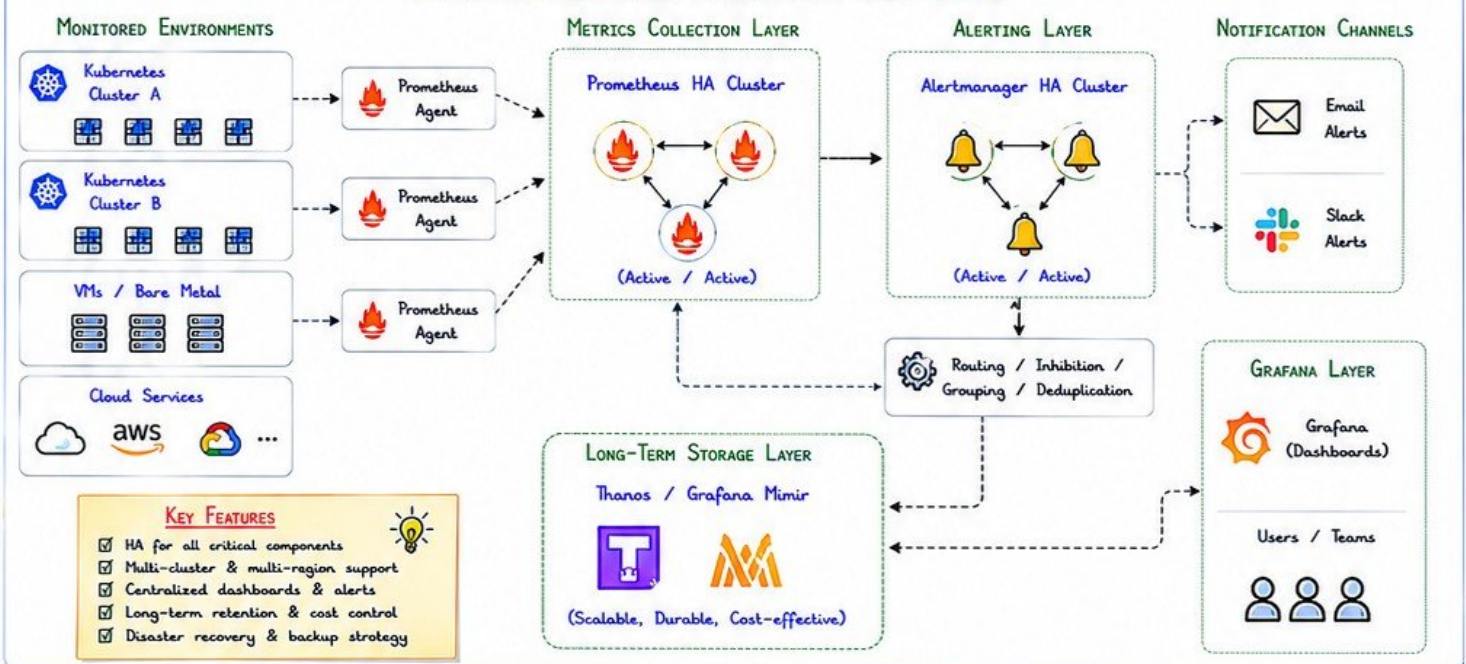


GOAL

Design, build and validate a production-ready monitoring platform that is scalable, highly available and disaster resilient.



ENTERPRISE MONITORING ARCHITECTURE (HIGH LEVEL)



LAB OBJECTIVES

- 1 Deploy Prometheus HA using Operator
- 2 Deploy Alertmanager HA
- 3 Deploy Grafana and configure dashboards
- 4 Integrate Thanos / Grafana Mimir for long-term storage
- 5 Monitor Kubernetes clusters (multi-cluster)
- 6 Configure email & Slack notifications
- 7 Setup backup & disaster recovery
- 8 Validate the entire monitoring platform

STEP-BY-STEP LAB FLOW

- Step 1 Provision infrastructure (K8s / Cloud)
- Step 2 Install Prometheus Operator
- Step 3 Deploy Prometheus HA (3 replicas)
- Step 4 Deploy Alertmanager HA (3 replicas)
- Step 5 Install Grafana & import dashboards
- Step 6 Integrate Thanos / Mimir (object storage) for long-term storage
- Step 7 Configure notification channels (Email, Slack)
- Step 8 Test, validate & document everything

WHAT YOU WILL BUILD

- Highly Available Monitoring Platform
- Centralized Observability Stack
- Cross-cluster Metrics Visibility
- Reliable Alerting with HA
- Long-term Storage & Query
- Actionable Dashboards
- Disaster Recovery Ready



DISASTER RECOVERY PLAN

- Back up Prometheus data (remote_write to object storage)
- Alertmanager configuration backup
- Regular snapshot of Grafana (dashboards, datasources)
- Infrastructure as Code (IaC) for quick rebuild
- Cross-region replication for object storage
- Run periodic DR drills (restore & failover tests)

VALIDATION CHECKLIST

- All targets are UP
- Alerts are firing & resolving correctly
- Email & Slack notifications received
- Dashboards load metrics correctly
- Long-term queries work (e.g., 30d+)
- Failover test successful
- Backups tested & restores verified
- Documentation completed

DELIVERABLES

- Architecture diagram
- Kubernetes manifests / Helm values
- Dashboards (JSON)
- Alert rules & notification configs
- Runbooks & documentation
- DR plan document
- Demo & validation report

SUCCESS CRITERIA

The monitoring platform is highly available, scalable, reliable and ready for production!



TIP:

Monitoring is not a one-time setup. Continuously improve, iterate and automate!

“Great monitoring doesn't just detect problems, it helps you prevent them.”
— SRE Mindset



FINAL NOTE

You've completed the Advanced Prometheus journey! Keep building, keep observing, keep improving. You are now a Monitoring Expert!

